

Sprint 1 — Answer Sheet

Orientation, Foundational Technologies & AI Tools

20 questions • Answer key derived directly from the XML's fields

Q1. You are given access to a shared Linux server that hosts a team's build tools, and you create a small notes file containing some temporary credentials for a test account. A long listing shows the permission string -rw-r--r--. You want to lock this down so that only you can read or write it, and no one else — including your own group — has any access at all. Which command achieves exactly this?

Single answer

CORRECT ANSWER

- ✓ `chmod 600 notes.txt`

Q2. A colleague asks you to find the one line reporting an ERROR in a large application log file on a Linux server, and to tell them the class and line number it references, without opening the whole file in an editor. Which of the following approaches would correctly locate it? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ `grep "ERROR" app.log`
- ✓ `awk '/ERROR/' app.log`
- ✓ Open the file with `less app.log`, then search forward with `/ERROR`

Q3. You are troubleshooting a service on a Linux server and use `tail -f app.log` to watch new log lines appear in real time while another process writes to it. You now want to stop watching and return to a normal shell prompt, without closing your SSH session or stopping the service itself. What should you do?

Single answer

CORRECT ANSWER

- ✓ Press `Ctrl+C` to interrupt the `tail -f` command

Q4. On your Windows laptop, you want to check whether a local PostgreSQL instance is currently running as a Windows service, and separately want to confirm the value of the `JAVA_HOME` environment variable, both from a PowerShell prompt rather than the GUI. Which pair of cmdlets is the correct fit for these two checks?

Single answer

CORRECT ANSWER

- ✓ `Get-Service` for the service check, and `Get-ChildItem Env:` (or `$env:JAVA_HOME`) for the environment variable

Q5. You run `Get-Process | Where-Object { $_.Name -like "**java*" }` in PowerShell and get back full process objects with named properties you can filter and sort on directly, whereas a colleague working on a Linux box runs `ps aux | grep java` and gets back plain lines of text that would need further parsing to extract just the process name or ID programmatically. Which of the following statements correctly describe this difference between PowerShell's pipeline and a traditional bash pipeline? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ PowerShell passes structured objects with properties between pipeline stages, rather than plain text
- ✓ A PowerShell pipeline stage can access a specific property directly (e.g. `$_ .Id`) without needing to parse or extract it from a line of text
- ✓ A bash pipeline stage typically has to parse plain text output (e.g. with `awk` or `cut`) to extract a specific field

Q6. Your team currently permits GitHub Copilot only as a learning aid while newer engineers get up to speed with an unfamiliar Jenkinsfile — every prompt should ask it to explain something, not to write or complete code for them. Which of the following prompts fit that guardrail? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ "Explain what each stage in this Jenkinsfile does, in plain English"
- ✓ "What does the 'post { always { ... } }' block typically get used for in a Jenkins pipeline?"
- ✓ "Why might a team choose a Multibranch Pipeline job over a plain Pipeline job — explain the difference"

Q7. You ask GitHub Copilot Chat to explain what a particular Jenkinsfile option does before you change it, and it gives a confident, detailed answer. Following good practice for using GenAI as a learning aid, what should you do next before relying on that explanation in a change you're about to make?

Single answer

CORRECT ANSWER

- ✓ Check the specific claim against a second source, such as the official documentation or a teammate, and note whether it was accurate, partially accurate, or wrong

Q8. You're setting up a new repository for a small internal tool. The project folder contains a README.md, a src/ directory of source files, a target/ folder holding compiled build output, and an .idea/ folder holding IntelliJ's per-user workspace state. You want your very first commit to include only the files that belong in version control. What is the correct sequence?

Single answer

CORRECT ANSWER

- ✓ Create a .gitignore excluding target/ and .idea/ before staging, then stage and commit only the source files

Q9. You've already committed a project's initial source files, and now want to add a .gitignore excluding target/ and .idea/. A colleague suggests just folding this into the next feature commit "to save a commit." Which of the following are genuine reasons to commit the .gitignore on its own instead? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ It keeps the commit history readable, since each commit then represents one coherent change rather than mixing tooling changes with feature changes
- ✓ It makes the .gitignore change easy to find, review, or revert independently of unrelated feature work
- ✓ A colleague scanning the log later can tell at a glance what the .gitignore commit was for, without needing to also read an unrelated feature diff

Q10. You create a feature branch from main, commit one small change, switch back to main, and merge the branch in. Running git log --oneline --graph --all afterwards shows a single straight line of commits with no separate merge commit. Which statements about what just happened are true? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ This was a fast-forward merge, because main had not moved since the feature branch was created
- ✓ Git simply moved the main pointer forward to the feature branch's latest commit, rather than creating a new merge commit
- ✓ If main had received other commits in the meantime, this same merge would likely have needed a real merge commit instead

Q11. You and a colleague have each branched off the same file to make different wording changes to a shared configuration message, and both branches change the exact same line. When you merge the second branch into main, Git reports a conflict and inserts <<<<<<, =====, and >>>>>> markers into the file. What is the correct next step to resolve this?

Single answer

CORRECT ANSWER

- ✓ Edit the file to keep the agreed wording, remove all three types of conflict markers, then stage the resolved file and complete the merge commit

Q12. A small team of 6-8 engineers shares a single repository and wants to release frequently, in small increments, without maintaining long-lived develop, release, or hotfix branches alongside main. Which of the following statements about trunk-based development make it a good fit here? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ Feature branches are kept short-lived and are merged back into main frequently, reducing the size and risk of each merge
- ✓ It avoids the overhead of maintaining several long-lived branches (develop, release, hotfix) alongside main
- ✓ Frequent small merges suit a team that wants to release often, since large batches of change aren't left to accumulate on a branch

Q13. You're reviewing a CI pipeline log with a colleague. It shows four stages running in order — Checkout, Build, Test, Archive — and the pipeline stops after Archive with no deployment step. Based on the distinction between Continuous Integration, Continuous Delivery, and Continuous Deployment, how should this pipeline be classified, and what would need to change to make it Continuous Deployment?

Single answer

CORRECT ANSWER

- ✓ It is Continuous Integration (with delivery groundwork via Archive); Continuous Deployment would require an automated step that deploys the archived artifact to an environment without manual intervention

Q14. You're asked to predict what would happen to a pipeline's later stages if its Build stage failed to compile the project, given that each stage in a typical Jenkins pipeline depends on the artifacts produced by the one before it. What would you expect to happen to the subsequent Test and Archive stages?

Single answer

CORRECT ANSWER

- ✓ Test and Archive would not run — a failed Build stage stops the pipeline, since there is no compiled artifact for later stages to test or archive

Q15. A colleague's Jenkins Pipeline job runs a Jenkinsfile whose Test stage executes mvn -B test successfully, but the post-build step junit 'target/test-reports/*.xml' then fails, with Console Output reporting it cannot find any matching files. A teammate suggests just deleting the junit step so the build goes green. Which statements about diagnosing and responding to this are correct? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ Deleting the junit step would make the build pass, but it would also stop reporting individual test results, hiding future test failures instead of fixing the underlying path mismatch
- ✓ The right next step is finding out where Maven actually wrote the XML reports — e.g. by running the build locally and inspecting the target/ folder — then pointing the junit step at that real path
- ✓ A build going "green" by removing a reporting step rather than fixing the mismatch is a false signal of health, since the pipeline no longer verifies anything about test results

Q16. Before trusting GitHub Copilot's explanation of why a Jenkins Test stage is failing, you decide to check the explanation for accuracy by running the equivalent command locally and inspecting what actually gets created on disk. Which local command lets you directly verify a claim about where Maven Surefire writes its XML reports?

Single answer

CORRECT ANSWER

- ✓ mvn test, then inspecting the contents of the target/ directory afterwards

Q17. You're reviewing a Dockerfile that has two FROM lines and are asked to explain why it produces a smaller final image than writing the whole build in a single stage. Which of the following statements correctly describe what a multi-stage Docker build does? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ An earlier stage (e.g. one containing the JDK and build tools) can produce build artifacts that a later stage then copies in with COPY --from
- ✓ The final image only needs to contain the runtime and the built artifact, not the compiler or build toolchain used to produce it
- ✓ Choosing a minimal, JRE-only base image for the final stage (rather than a full JDK image) further reduces the final image's size

Q18. You've written a single-stage Dockerfile starting from eclipse-temurin:21-jre-alpine, copied in an already-built jar, and set it as the entrypoint. After building the image and running a detached, named container, you want to confirm the jar file is actually present inside the running container's filesystem. Which command lets you get an interactive shell inside the running container to check?

Single answer

CORRECT ANSWER

- ✓ docker exec -it sh

Q19. You are new to the team and, while shadowing a release, notice that after a colleague rebuilds a Docker image using the same tag as a previous build, docker images now shows an additional entry tagged . Which of the following statements about this are correct? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ It is the old image left behind because the tag moved to the newly built image, leaving the previous image untagged
- ✓ It can be safely cleaned up with docker image prune, as long as nothing still depends on it
- ✓ A running container started from that old image ID would keep working even after its tag was reassigned, since containers reference a specific image ID, not just a tag

Q20. You're pairing with a new starter who is still getting familiar with the team's standard toolchain and practices. Which of the following statements would you confirm as accurate when explaining how these tools and practices fit together? (Select all that apply.)

Select all that apply

CORRECT ANSWERS

- ✓ GitHub Copilot is best used as a learning aid for explaining unfamiliar code and error output before it's trusted to generate new code unsupervised
- ✓ A .gitignore file should be created and committed before build output or IDE-specific folders are ever staged, so they never enter the repository's history in the first place
- ✓ Trunk-based development, using short-lived feature branches merged frequently, suits a small team releasing often better than a long-lived Gitflow-style branch set
- ✓ A CI pipeline that stops after building, testing, and archiving an artifact — without deploying it anywhere automatically — is Continuous Integration, not Continuous Deployment